

Finite Horizon Dynamic Programming for Hydro Reservoirs

Christopher Won — **Master 2 Optimization**
École Polytechnique, Université Paris-Saclay
 christopher.won@polytechnique.edu

Abstract

WE consider the stochastic optimal control of a small valley of hydroelectric dams, based on the model of Carpentier et al.^[1]. The continuous state space, representing the water level in each dam, is discretized into a finite grid, reducing the problem to a Markov Decision Process (MDP) with finite-horizon criteria. We implement the Bellman operator backward induction to compute the value function and the optimal feedback control on valleys of 2, 4 and 6 dams. They publish no instance data for their academic valleys, so no numerical comparison is possible; we reproduce their topologies and measure instead what the method's two approximations cost: a finite grid of volumes and a finite set of releases. Multilinear interpolation attains on 121 grid points the accuracy nearest-neighbour rounding reaches only on 103,041, and to fail on the safe side: the interpolated model predicts slightly less revenue than its own strategy delivers, whereas the rounded model promises more than it delivers. On a six-dam valley the discretization error alone, 1.3%, is comparable in magnitude to the 3% by which the DADP and SDDP values^[1] differ there.

I. INTRODUCTION

HYDRO valley management is an application of stochastic optimal control where a cascade of dams stores water that is released through turbines to generate electricity sold on the market. However, the natural (environmental) inflow of water into each dam and the electricity price are random at each time step^[1]. This makes how much water to release from each dam, at each time step, to maximize expected profit or equivalently, to minimize the total cost, a complex decision-making problem. Moreover, this is made more difficult by the coupling between dams where water released or spilled from an upstream dam becomes part of the inflow of the next one.

This paper follows the work of Carpentier et al.^[1], who addressed this problem for large-scale hydro valleys using Stochastic Dual Dynamic Programming (SDDP)^[2] and more relevantly, Dual Approximate Dynamic Programming (DADP)^[3]. DADP is built around Lagrangian relaxation of the coupling constraints between dams. However, we do not implement DADP, nor any other decomposition or approximation scheme. Instead, we consider a small-scale hydro valley, discretize the (originally continuous) state space, and implement the exact finite-horizon Bellman operator by backward induction. This is the same approach the article uses as a reference solution on small valleys, where it remains tractable^[1].

The remainder of this paper is organized as follows: Section II formulates the discretized Markov decision process;

Section III describes the backward induction algorithm; Section IV reports numerical results and sets them against the baseline^[1]; Section V states the limits of the approach and concludes.

II. MODEL

TO begin, a hydro valley is a set of hydroelectric dams on the same river system, operated jointly rather than independently because they are connected by water flow: each dam stores water in a reservoir, turbines a portion of it to produce electricity, and spills the excess when the reservoir is full. Then, the released water flows into the reservoir directly downstream^[1].

A. Continuous-State model

1) State, dynamics and controls: We consider a hydro valley made of d dams, indexed by $i \in \{1, \dots, d\}$. The optimization horizon is one year, discretized into $T = 12$ monthly time steps^[1]: decisions are taken at $t \in \{0, \dots, T-1\}$ and the trajectory ends at X_T . At each time step t , the state of the system is the vector collecting the water volume of every dam:

$$X_t = (X_t^1, \dots, X_t^d) \in \mathcal{X} := \prod_{i=1}^d [\underline{x}^i, \bar{x}^i] \subset \mathbb{R}^d, \quad (1)$$

where X_t^i is the volume of water stored in dam i , and the initial volumes $X_0 = x_0$ are known. Each coordinate ranges continuously over $[\underline{x}^i, \bar{x}^i]$, the admissible volume interval of dam i , where $\bar{x}^i$ and $\underline{x}^i$ are, respectively, the maximum capacity and minimal volume of dam i . The state space $\mathcal{X}$ is infinite despite d being a small, fixed number of dams ($d \in \{2, 3, 4, 5, 6\}$). Each of its d coordinates can take uncountably many values in $\mathbb{R}$, meaning the state space is not compatible with the finite or discrete state space E required by Definition 3.1^[4]. This motivates the discretization^{[1], [5]} introduced later in this section, which replaces $\mathcal{X}$ with a finite set E .

For each dam $i \in \{1, \dots, d\}$, let U_t^i denote the volume of water turbinated (released) of dam i at time t . The exogenous noise affecting dam i at time t is the pair $W_t^i = (Q_t^i, P_t^i)$ with $Q_t = (Q_t^1, \dots, Q_t^d)$ and $P_t = (P_t^1, \dots, P_t^d)$, where Q_t^i is the environmental inflow of water into dam i (rain, snowmelt, tributaries) and P_t^i is the market price of the energy produced at dam i ^[1]. The inflows are water volumes, hence $Q_t^i \geq 0$; the prices P_t^i are unrestricted in sign. We write $W_t = (W_t^1, \dots, W_t^d)$, assumed independent across

$t = 0, \dots, T-1$, and independent of X_0 , taking values in a finite set $\mathbb{W} \subset \mathbb{R}_+^d \times \mathbb{R}^d$, whose elements are the pairs $w = (q, p)$ of an inflow vector and a price vector the noise can take; enumerated in (46) and fixed for our instance in Section IV-A, so that the process $(X_t)_{t \geq 0}$ defined below is a controlled Markov chain (Fact 2.11^[4]).

The topology of the hydro valley is described as follows: each dam flows into at most one dam directly downstream, and we denote by $\mathcal{P}(i)$ the set of dams flowing directly into dam i , with $\mathcal{P}(i) = \emptyset$ for an uppermost dam^[1]. Dams are indexed so that $j < i$ for all $j \in \mathcal{P}(i)$, i.e. every dam has a larger index than all the dams upstream of it.

As in^[1], the volume spilled when dam i overflows is:¹

$$s_t^i(x, u, w) = \max(0, x^i - u^i + q^i + z_t^i(x, u, w) - \bar{x}^i), \quad (2)$$

where $z_t^i(x, u, w)$ is the total outflow (turbinated + spilled water) of the dams directly upstream² of dam i :

$$z_t^i(x, u, w) = \sum_{j \in \mathcal{P}(i)} (u^j + s_t^j(x, u, w)), \quad (3)$$

with $z_t^i \equiv 0$ when $\mathcal{P}(i) = \emptyset$. A linear cascade, in which the dams form a single chain with dam i flowing into dam $i+1$, corresponds to $\mathcal{P}(i) = \{i-1\}$ for $i \geq 2$. By the choice of indexing, (2)–(3) can be evaluated for $i = 1, \dots, d$ in order. Therefore, as in^[1], each dam's state evolves as:

$$f_t^i(x, u, w) = x^i - u^i + \underbrace{q^i}_{\text{environmental inflow}} + \underbrace{z_t^i(x, u, w) - s_t^i(x, u, w)}_{\text{upstream outflow}}, \quad (4)$$

and the dynamics of the valley are $X_{t+1} = f_t(X_t, U_t, W_t)$ with $f_t = (f_t^1, \dots, f_t^d)$; f_t depends on W_t only through the inflows $Q_t = (Q_t^1, \dots, Q_t^d)$. Note that z_t^i enters the dynamics at time t rather than $t+1$: water released or spilled by a dam during a month is assumed to reach the dam downstream within that same month, the transit time being negligible at a monthly resolution.

We adopt a Hazard-Decision (HD)^[1] information structure: U_t is chosen once W_t has been observed, so the admissible controls depend on both the state x and the noise w . The admissible control set $C_t^c(x, w)$ is the set of $u = (u^1, \dots, u^d)$ such that, for all i ,

$$u^i \in [\underline{u}^i, \min(\bar{u}^i, x^i + q^i + z_t^i(x, u, w) - \bar{x}^i)], \quad (5)$$

where $\underline{u}^i, \bar{u}^i$ are the minimal and maximal turbinated volumes of dam i , with $\underline{u}^i = 0$ throughout; the second term of the minimum, as in^[1], prevents turbinating more water than is available above the minimal volume $\bar{x}^i$. This set is well-defined since the bound on u^i involves z_t^i , which depends only on u^j with $j < i$; the admissible values can therefore be

¹Lowercase letters denote generic realizations of the corresponding random variables: x of the state X_t , u of the control U_t , and $w = (q, p)$ of the noise $W_t = (Q_t, P_t)$.

²In [1], the upstream inflow Z_t^i of dam i is treated as an additional control of dam i , since the coupling between dams is dualized to decompose the valley dam by dam. As no decomposition is performed here, z_t^i is simply computed from (x, u, w) through (3).

determined sequentially from $i = 1$ to d . The controls must then satisfy $U_t \in C_t^c(X_t, W_t)$. At a monthly resolution, the HD framework grants the decision maker knowledge of the month's environmental inflow q before choosing the month's release u . It is justified in^[1] by the fact that the monthly time step is an aggregation of a finer operation where the release is implemented daily. The manager adjusts throughout the month as inflows are observed, so that the aggregate monthly release depends on the realized monthly inflow. The alternative Decision-Hazard convention, in which U_t is chosen before W_t is revealed, is conversely too pessimistic as it prevents any reaction within the month. The value computed here is therefore an optimistic estimate of what could be achieved by an operator deciding daily^[6].

2) Running and terminal costs: The running cost at time t is the sum over the dams of the cost of each,

$$c_t(x, u, w) = \sum_{i=1}^d (-p^i u^i + \varepsilon (u^i)^2), \quad (6)$$

where p^i is the price of the energy produced at dam i . The first term is the opposite of the revenue obtained by selling the energy produced from the volume u^i , the criterion being written as a minimization^[1]. The second term, with $\varepsilon \geq 0$, charges the cost of operating the water turbines, which grows with the volume passed through them. Being convex in u , it also supplies the strong convexity that the decomposition methods require^[1], a property the approach taken in this paper does not need. Nonetheless, we keep the term for the first reason, with $\varepsilon = 10^{-3}$ in Table I.

At the final time T , a terminal cost³ penalizes dams left below a target volume,

$$\phi(x_T) = \sum_{i=1}^d \alpha^i \max(0, \hat{x}^i - x_T^i)^2, \quad (7)$$

where $\hat{x}^i \in [\underline{x}^i, \bar{x}^i]$ is the desired volume of dam i at the end of the horizon and $\alpha^i > 0$ weights the penalty.

3) Optimization problem: The total additive cost with finite horizon T is the expected total cost over the horizon:

$$J^{(T, \sigma)} = \mathbb{E} \left[\sum_{t=0}^{T-1} c_t(X_t, U_t, W_t) + \phi(X_T) \right], \quad (8)$$

and the optimization problem to be solved is the minimization of (8) over the strategies σ of Definition 3.10^[4]:

$$\min_{\sigma} J^{(T, \sigma)} \quad (9)$$

subject to, for $t = 0, \dots, T-1$,

$$X_{t+1} = f_t(X_t, U_t, W_t), \quad X_0 = x_0, \quad (10)$$

$$U_t \in C_t^c(X_t, W_t), \quad (11)$$

the expectation being taken over the noise $(W_t)_{0 \leq t \leq T-1}$. We define strategy $\sigma = (\sigma_t)_{0 \leq t \leq T-1}$ as a sequence of rules giving the control at each time step from the information available then i.e., pure and history-dependent (non-Markovian). Once

³[1] writes $\min\{0, \hat{x}^i - x_T^i\}^2$, which penalizes volumes *above* the target and leaves shortfalls free of penalty.

σ is fixed, the trajectory is generated from $X_0 = x_0$ by $U_t = \sigma_t(X_0, U_0, \dots, X_{t-1}, U_{t-1}, X_t, W_t)$ and the dynamics (4), so $(X_t, U_t)_{0 \leq t \leq T}$ is a deterministic function of the noise, the function itself being determined by σ . The law of (X_t, U_t) , and hence the criterion, therefore depends on σ alone.

B. Grid discretization

The state space $\mathcal{X}$ is now replaced by a finite set, following the approach of discretizing the state and control spaces and solving the resulting finite-dimensional problem^{[2], [7]}. Each interval $[\underline{x}^i, \bar{x}^i]$ is discretized onto its uniform one-dimensional grid on dam i of n points⁴:

$$\mathcal{G}^i = \{\underline{x}^i + k \delta^i : k = 0, \dots, n-1\}, \quad \delta^i = \frac{\bar{x}^i - \underline{x}^i}{n-1}, \quad (12)$$

where δ^i is the spacing between two consecutive grid points of dam i ; both endpoints belong to $\mathcal{G}^i$, so the n points define $n-1$ intervals of equal length. Denote the points of the grid $\mathcal{G}^i$ by:

$$g_{k_i}^i := \underline{x}^i + k \delta^i, \quad k = 0, \dots, n-1. \quad (13)$$

The discretized state space is therefore the (product) grid:

$$E = \mathcal{G}^1 \times \dots \times \mathcal{G}^d, \quad |E| = n^d. \quad (14)$$

The points of E divide $\mathcal{X}$ into $(n-1)^d$ cells: the boxes $\prod_{i=1}^d [g_{k_i}^i, g_{k_i+1}^i] \subset \mathbb{R}^d$ whose vertices are points of E , and every point of $\mathcal{X}$ belongs to at least one of them.

The admissible controls are discretized in similar fashion: each $[\underline{u}^i, \bar{u}^i]$ is replaced by the uniform grid $\mathcal{U}^i$ of m points which, as $\mathcal{G}^i$ in (12), contains both endpoints, and from here on $C_t(x, w)$ denotes the finite set:

$$C_t(x, w) = C_t^c(x, w) \cap (\mathcal{U}^1 \times \dots \times \mathcal{U}^d) \quad (15)$$

of grid controls satisfying (5).

C. Interpolation rule

For $x \in E$, the next state $f_t(x, u, w)$ remains in the box $\mathcal{X} = \prod_{i=1}^d [\underline{x}^i, \bar{x}^i]$: the spillage (2) removes any volume above $\bar{x}^i$, and the admissible set (5) prevents releasing more water than is available above $\underline{x}^i$, so the next state never has to be forced back into the admissible range. However, it also does not belong to E necessarily as reaching a grid point would require the net volume change $-u^i + q^i + z_t^i - s_t^i$ to be an exact multiple of the grid step δ^i ; the model does not enforce this. A rule is required to map the dynamics $f_t(x, u, w)$ back onto E .

The following two rules were considered. Nearest-neighbour rounding replaces the next state by the closest grid point, discarding the discrepancy. **Multilinear interpolation** distributes the next state over the grid points surrounding it, in proportion to its position among them^[8]. We use the latter, for reasons given in Section II-C5.

⁴In [1], inflows and turbine controls are discrete, so that each dam is modeled as a discrete Markov chain; the grid resolution and the discretization step are not reported, and no interpolation rule is mentioned. The uniform grid below follows the discretization approach of [7], and the multilinear interpolation of Section II-C is covered by the analysis of [8].

1) Cell index: Fix $x \in E$, an admissible control u and a noise realization w , and write $y = f_t(x, u, w) \in \mathcal{X}$ for the resulting next state. In each coordinate i , let

$$k_i = \min \left(\left\lceil \frac{y^i - \underline{x}^i}{\delta^i} \right\rceil, n-2 \right) \quad (16)$$

be the index of the grid interval containing y^i , so that

$$g_{k_i}^i \leq y^i \leq g_{k_i+1}^i, \quad (17)$$

and let

$$\lambda^i = \frac{y^i - g_{k_i}^i}{\delta^i} \in [0, 1] \quad (18)$$

be the position of y^i within that interval (cell), expressed as a fraction of its length. Thus $\lambda^i = 0$ when y^i coincides with the left endpoint $g_{k_i}^i$, $\lambda^i = 1$ when it coincides with the right endpoint $g_{k_i}^i + \delta^i = g_{k_i+1}^i$, and intermediate values interpolate between the two. Equivalently, λ^i is the coefficient expressing y^i as a convex combination of its two neighbours:

$$(1 - \lambda^i) g_{k_i}^i + \lambda^i g_{k_i+1}^i = y^i. \quad (19)$$

Both truncations in (16) need to be explicitly addressed. The truncation at $n-2$ applies only when $y^i = \bar{x}^i$, for which the floor returns $n-1$ and the cell $[g_{n-1}^i, g_n^i]$ does not exist; it selects the last cell instead, with $\lambda^i = 1$. No truncation from below appears because u is admissible: (5) gives $x^i - u^i + q^i + z_t^i \geq \underline{x}^i$, so $y^i \geq \underline{x}^i$ whether or not dam i spills, and the floor is nonnegative.

2) Vertices of the cell: (17) holds in every coordinate, so $y = (y^1, \dots, y^d)$ lies in the cell $\prod_{i=1}^d [g_{k_i}^i, g_{k_i+1}^i]$, of side lengths $\delta^1, \dots, \delta^d$, whose vertices are obtained by choosing, independently for each dam i , either the left endpoint $g_{k_i}^i$ or the right one $g_{k_i+1}^i$. Geometrically, there are two choices in each of d coordinates (dams) which yield 2^d vertices. We index them by the vertex set of the unit hypercube: for $\kappa = (\kappa^1, \dots, \kappa^d) \in \{0, 1\}^d$, let

$$g_\kappa = (g_{k_1+\kappa^1}^1, \dots, g_{k_d+\kappa^d}^d) \in E, \quad (20)$$

so that $\kappa^i = 0$ selects the left neighbour in coordinate i and $\kappa^i = 1$ the right one. The entries of κ record which neighbour is selected for each dam; they are not water volumes. The vertex g_κ is a vector of volumes in the state space $E \subset \mathcal{X}$, whereas κ ranges over $\{0, 1\}^d$.

3) Weights: Each dam contributes one factor to the weight of a vertex: λ^i if $\kappa^i = 1$, that is if the right neighbour was selected in coordinate i , and $1 - \lambda^i$ if $\kappa^i = 0$. Since the selection is made independently in each dam, the weight assigned to g_κ is the product of these d factors,

$$\omega_\kappa(y) = \prod_{i=1}^d (\kappa^i \lambda^i + (1 - \kappa^i)(1 - \lambda^i)), \quad (21)$$

where the i -th factor reduces to λ^i when $\kappa^i = 1$ and to $1 - \lambda^i$ when $\kappa^i = 0$. The index κ is not chosen: (21) is evaluated for all 2^d values of κ , and every vertex of the cell contributes. This is what distinguishes interpolation from rounding, where a single vertex would receive the whole mass.

These weights are nonnegative as products of numbers in $[0, 1]$, and expanding the product over the 2^d terms gives

$$\sum_{\kappa \in \{0,1\}^d} \omega_\kappa(y) = \prod_{i=1}^d (\lambda^i + (1 - \lambda^i)) = 1. \quad (22)$$

Moreover, substituting (19) coordinate by coordinate,

$$\sum_{\kappa \in \{0,1\}^d} \omega_\kappa(y) g_\kappa = y. \quad (23)$$

4) Interpretation: The family $(\omega_\kappa(y))_\kappa$ admits two equivalent readings.

Read as interpolation coefficients, the weights are a convex combination by (22), and they extend a function known only on the grid to points lying between grid points^[2]. Let $v : E \rightarrow \mathbb{R}$ be such a function: its values are stored only at the n^d points of E , whereas $y = f_t(x, u, w)$ is not one of them necessarily. Its value at y is then approximated by the weighted average of its values at the 2^d vertices of the cell containing y ,

$$v(y) \approx \sum_{\kappa \in \{0,1\}^d} \omega_\kappa(y) v(g_\kappa), \quad (24)$$

each vertex being weighted according to the proximity of y to it. The sum runs over the 2^d vertices, not over the dams: each term $v(g_\kappa)$ is the single number obtained by evaluating v at one vertex $g_\kappa \in E$, the dams entering only through the product defining ω_κ in (21). Nothing in (24) is specific to the function v ; it is applied in Section II-D to the value functions of the backward recurrence, which are likewise computed and stored only at the points of E .

Read as a probability distribution over the 2^d vertices of the cell, (23) states that its expectation is the true next state y itself. This is the reading adopted here: the discretized model is not the continuous model with an interpolation step appended, but a Markov decision process in the sense of Definition 3.1^[4] (the derived MDP of Theorem 4.1^[8]), whose state space is the finite set $E \times \mathbb{W}$ and whose transitions land on the vertices of the cell containing $f_t(x, u, w)$ with probabilities ω_κ . The transition probabilities are written explicitly in Section II-D. The two readings coincide numerically, (24) being the expectation of v under that distribution, so the same computation serves both purposes.

5) Comparison with rounding: Nearest-neighbour rounding displaces the next state by up to $\delta^i/2$ in each coordinate, the worst case occurring when y^i falls at the midpoint of its interval. This displacement is biased: the rounded state does not coincide with $f_t(x, u, w)$ in expectation, and the discrepancy is propagated by the backward recursion. By (23), multilinear interpolation introduces no such bias. Section IV-C measures the difference: interpolation attains on $n = 11$ points per dam the accuracy rounding reaches only at $n = 321$, a factor of 29 in the resolution and hence of 852 in $|E|$ at $d = 2$. This matters directly here: the size of the state space is $|E| = n^d$, so any reduction in the resolution n needed for a given accuracy is amplified exponentially in the number of dams^{[2], [7]}.

The cost of this choice is that each transition reaches 2^d grid points instead of one. Evaluating (24) therefore requires

2^d products and additions per triple (x, u, w) , against a single term for rounding. This is a factor that grows with the number of dams and enters the complexity of the algorithm of Section III.

D. Discretized Markov decision process

1) State augmentation: The discretized model can now be written as a Markov decision process in the sense of Definition 3.1^[4]. One difficulty remains: that definition requires the admissible set $C_t(x)$ to depend only on the state, whereas under the Hazard-Decision structure (5) depends on the noise as well. This is resolved, as for problems with uncontrollable state components (Section 1.4^[5]; see also Exercise 2.3.1^[4]), by taking the pair

$$(X_t, W_t) \in E \times \mathbb{W} \quad (25)$$

as the state. The admissible set $C_t(x, w)$ then depends only on this augmented state, and $E \times \mathbb{W}$ is finite, E being finite by (14) and $\mathbb{W}$ by assumption.

2) Transition probabilities: For a model specified by a dynamics and an exogenous noise, Fact 3.9^[4] gives $M_{x,y}^{(t,u)} = \mathbb{P}(f_t(x, u, W_t) = y)$, where the probability averages over the noise. In the augmented model the noise is part of the state, hence conditioned on, and the volume term is instead $\mathbb{P}(f_t(x, u, W_t) = y \mid W_t = w) = \mathbf{1}_{\{y=f_t(x,u,w)\}}$, the dynamics being deterministic once w is fixed. This indicator is unusable here as it is nonzero only at $y = f_t(x, u, w)$, and whenever that point is not a grid point it vanishes at every $y \in E$, so the row sums to zero rather than one. The interpolation of Section II-C replaces it by the weights ω_κ , which spread the same mass over the 2^d vertices of the cell containing the target point $\tilde{y} = f_t(x, u, w)$. Indexing these weights by the grid point rather than by the vertex label,

$$\omega(y \mid \tilde{y}) = \begin{cases} \omega_\kappa(\tilde{y}) & \text{if } y = g_\kappa \text{ for some } \kappa \in \{0,1\}^d, \\ 0 & \text{otherwise,} \end{cases} \quad (26)$$

which is unambiguous since the vertices are distinct, so at most one κ satisfies $y = g_\kappa$. Equivalently in words, $\tilde{y}$ lies in a cell with 2^d vertices, and each vertex receives a weight given by the product over the dams of how close the target is to that vertex in each coordinate; a grid point y carries that weight if it is one of those vertices, and zero otherwise.

Since the next volume is one of the 2^d vertices, chosen at random, this choice needs its own source of randomness. Let $(\xi_t)_{1 \leq t \leq T}$ be random variables, uniformly distributed on $[0, 1]$, independent of each other, of X_0 and of $(W_t)_{0 \leq t \leq T-1}$, and let G be a map returning, from a target point and a value of ξ , one vertex of the cell containing that target, in such a way that the vertex y is returned with probability $\omega(y \mid \tilde{y})$:

$$\mathbb{P}(G(\tilde{y}, \xi_{t+1}) = y) = \omega(y \mid \tilde{y}), \quad \tilde{y} \in \mathcal{X}, y \in E. \quad (27)$$

The volume component of the state at time $t + 1$ is then

$$Y_{t+1} = G(f_t(X_t, U_t, W_t), \xi_{t+1}), \quad (28)$$

and the state at time $t + 1$ is the pair (Y_{t+1}, W_{t+1}) . Note that Y_{t+1} is the discretized counterpart of the volume X_{t+1}

of Section II-A1, the interpolation replacing the off-grid value $f_t(X_t, U_t, W_t)$ by a grid point.

Abbreviating $\mathbb{P}_{x,w,u}(\cdot) = \mathbb{P}(\cdot \mid X_t = x, W_t = w, U_t = u)$, the transition probabilities are:

$$\begin{aligned} M_{(x,w),(y,w')}^{(t,u)} &= \mathbb{P}_{x,w,u}(Y_{t+1} = y, W_{t+1} = w') \\ &= \mathbb{P}_{x,w,u}(Y_{t+1} = y) \\ &\quad \times \mathbb{P}_{x,w,u}(W_{t+1} = w' \mid Y_{t+1} = y) \\ &= \omega(y \mid \tilde{y}) \mathbb{P}(W_{t+1} = w'). \end{aligned} \quad (29)$$

The second equality is the chain rule and requires nothing. In the third, the first factor follows from (28) and the defining property of G , since $f_t(x, u, w)$ is determined by the conditioning; the second uses that W_{t+1} is independent of X_0 , of $W_0, \dots, W_t$ and of ξ_{t+1} , while X_t, U_t and Y_{t+1} are all functions of these, so that the conditioning is irrelevant. These transition probabilities are nonnegative and sum to one over $(y, w') \in E \times \mathbb{W}$, by (22) and because the law of W_{t+1} is a probability distribution. The discretized model is therefore a Markov decision process in the sense of Definition 3.1^[4], with finite state space $E \times \mathbb{W}$.

3) Dynamic programming equation: Following Section 3.2^[4], the criterion of the augmented problem started at time t in the state (x, w) is:

$$J_t^{(T,\sigma)}(x, w) = \mathbb{E} \left[\sum_{s=t}^{T-1} c_s(X_s, U_s, W_s) + \phi(X_T) \mid X_t = x, W_t = w \right] \quad (30)$$

in the form of (3.4)^[4], and the value function is, as in Definition 3.11^[4],

$$v_t(x, w) = \min_{\sigma} J_t^{(T,\sigma)}(x, w), \quad (x, w) \in E \times \mathbb{W}. \quad (31)$$

The costs c_t take finitely many values, since $E, \mathbb{W}$ and the discretized admissible sets $C_t(x, w)$ are all finite; c_t and ϕ are therefore bounded. The minimum in (32) below is over a finite set, which is moreover nonempty, as Definition 3.2 and Section 3.3^[4] require of every admissible set: the control $u = 0$ belongs to $C_t(x, w)$ for every $(x, w) \in \mathcal{X} \times \mathbb{W}$, since $0 \in \mathcal{U}^i$ and, by induction on i , $z_t^i \geq 0$ (each $w^j = 0$, and $s_t^j \geq 0$ by (2)), so that $x^i + q^i + z_t^i - \underline{x}^i \geq 0 = u^i$ in (5), using $x^i \geq \underline{x}^i$ and $q^i \geq 0$. The minimum is therefore attained. The hypotheses of Theorem 3.13^[4] are thus satisfied and so v satisfies the following backward recurrence, or Bellman dynamic programming equation:

$$\begin{aligned} v_t(x, w) &= \min_{u \in C_t(x, w)} \left\{ c_t(x, u, w) \right. \\ &\quad \left. + \sum_{(y, w') \in E \times \mathbb{W}} M_{(x,w),(y,w')}^{(t,u)} v_{t+1}(y, w') \right\}, \end{aligned} \quad (32)$$

for all $(x, w) \in E \times \mathbb{W}$ and $0 \leq t \leq T-1$, with final condition

$$v_T = \phi, \quad (33)$$

and the feedback policy $\pi = (\pi_t)_{0 \leq t \leq T-1}$ attaining the minimum in (32) is an optimal strategy of the problem.

(32) is a recursion on $E \times \mathbb{W}$. It reduces, as in Eq. (1.15)^[5], to a recursion on E alone, which is what is computed in

practice. Define the noise-averaged value function, for all $y \in E$, as:

$$\bar{v}_t(y) = \mathbb{E}[v_t(y, W_t)] = \sum_{w' \in \mathbb{W}} \mathbb{P}(W_t = w') v_t(y, w'), \quad (34)$$

and fix $u \in C_t(x, w)$, with $\tilde{y} = f_t(x, u, w)$. Then,

$$\begin{aligned} \sum_{(y, w') \in E \times \mathbb{W}} M_{(x,w),(y,w')}^{(t,u)} v_{t+1}(y, w') \\ = \sum_{y \in E} \sum_{w' \in \mathbb{W}} \omega(y \mid \tilde{y}) \mathbb{P}(W_{t+1} = w') v_{t+1}(y, w') \end{aligned} \quad (35)$$

$$= \sum_{y \in E} \omega(y \mid \tilde{y}) \sum_{w' \in \mathbb{W}} \mathbb{P}(W_{t+1} = w') v_{t+1}(y, w') \quad (36)$$

$$= \sum_{y \in E} \omega(y \mid \tilde{y}) \bar{v}_{t+1}(y) \quad (37)$$

$$= \sum_{\kappa \in \{0,1\}^d} \omega_{\kappa}(\tilde{y}) \bar{v}_{t+1}(g_{\kappa}), \quad (38)$$

where (35) substitutes (29), (36) uses that $\omega(y \mid \tilde{y})$ does not depend on w' , (37) applies (34) at time $t+1$, and (38) uses that $\omega(\cdot \mid \tilde{y})$ vanishes outside the 2^d vertices by (26). Substituting into (32),

$$\begin{aligned} v_t(x, w) &= \min_{u \in C_t(x, w)} \left\{ c_t(x, u, w) \right. \\ &\quad \left. + \sum_{\kappa \in \{0,1\}^d} \omega_{\kappa}(f_t(x, u, w)) \bar{v}_{t+1}(g_{\kappa}) \right\}, \end{aligned} \quad (39)$$

the sum over κ being the interpolation (24) applied to $\bar{v}_{t+1}$, which is known only on E . Applying (34) at time t to (39),

$$\begin{aligned} \bar{v}_t(x) &= \mathbb{E} \left[\min_{u \in C_t(x, W_t)} \left\{ c_t(x, u, W_t) \right. \right. \\ &\quad \left. \left. + \sum_{\kappa \in \{0,1\}^d} \omega_{\kappa}(f_t(x, u, W_t)) \bar{v}_{t+1}(g_{\kappa}) \right\} \right], \end{aligned} \quad (40)$$

for $t = T-1, \dots, 0$, with, from $v_T(x, w) = \phi(x_T)$ and $\sum_{w'} \mathbb{P}(W_T = w') = 1$ ⁵,

$$\bar{v}_T(x) = \sum_{w' \in \mathbb{W}} \mathbb{P}(W_T = w') \phi(x_T) = \phi(x_T). \quad (41)$$

The optimal policy is derived from (39),

$$\begin{aligned} \pi_t(x, w) &\in \arg \min_{u \in C_t(x, w)} \left\{ c_t(x, u, w) \right. \\ &\quad \left. + \sum_{\kappa \in \{0,1\}^d} \omega_{\kappa}(f_t(x, u, w)) \bar{v}_{t+1}(g_{\kappa}) \right\} \end{aligned} \quad (42)$$

and the value of the discretized Markov decision problem, that is the minimum (31) at $t = 0$ averaged over W_0 as in (34), is $\bar{v}_0(x_0)$ if $x_0 \in E$. Otherwise, the discretized problem is started from the initial law $\omega(\cdot \mid x_0)$ on E , as Definition 3.1^[4] allows, and its value is $\sum_{\kappa \in \{0,1\}^d} \omega_{\kappa}(x_0) \bar{v}_0(g_{\kappa})$, the interpolation (24) of $\bar{v}_0$ at x_0 .

⁵The noise W_T does not enter the dynamics, which stop at X_T . It is introduced only so that the augmented state (25) is defined at the final time; its law, any law on $\mathbb{W}$ independent of $X_0, W_0, \dots, W_{T-1}$, plays no role since $v_T(x, w) = \phi(x_T)$ for all w .

Two features of (40) are worth noting. First, the expectation is outside the minimum, as in equation (3b) of Carpentier et al.^[1], which is the HD structure: for each noise realization the decision maker solves a deterministic minimization, and the resulting values are averaged^[6]. Second, although the policy (42) is indexed by the pair (x, w) , only the n^d numbers $\bar{v}_t(y)$, $y \in E$, need be stored at each time step, rather than the $n^d \times |\mathbb{W}|$ values of v_t ^[5].

III. ALGORITHM

SECTION II produced a specification: the backward recurrence (40) with terminal condition (41), and the policy (42) attaining its minimum. It does not say in what order the quantities are evaluated, how the sets indexing them are enumerated, or what the resulting computation costs. This section supplies these, and states the recursion as an algorithm whose output is shown to coincide with $\bar{v}_t$. The numerical values obtained are reported in Section IV.

A. Indexing and notation

The recursion (40) acts on functions $\bar{v}_t : E \rightarrow \mathbb{R}$. Writing it as an algorithm requires explicit labels for the three sets over which it ranges: grid states, noises and controls.

1) Grid points as multi-indices: By (14), $E = \mathcal{G}^1 \times \dots \times \mathcal{G}^d$, and each $\mathcal{G}^i$ for $i = 1, \dots, d$ is enumerated by $k_i \in \{0, \dots, n-1\}$ through (12). A point of E is therefore determined by the vector of its per-dam indices. Let

$$\mathcal{K} := \{0, \dots, n-1\}^d, \quad g_k := (g_{k_1}^1, \dots, g_{k_d}^d), \quad (43)$$

for $k = (k_1, \dots, k_d) \in \mathcal{K}$. The map $k \mapsto g_k$ is a bijection from $\mathcal{K}$ onto E , being the product of the d bijections $k_i \mapsto g_{k_i}^i$, and $|\mathcal{K}| = n^d = |E|$.

This encompasses the vertex notation (20). If k denotes the vector of cell indices (16) of a target point $\tilde{y}$, then $k \in \{0, \dots, n-2\}^d$, so $k + \kappa \in \mathcal{K}$ for every $\kappa \in \{0, 1\}^d$, and the vertex written g_κ in (20) is $g_{k+\kappa}$ in the notation of (43). The label κ is thus an increment of the cell index. From here onwards, subscripts of g are elements of $\mathcal{K}$, and the vertices of Section II-C are written $g_{k+\kappa}$.

2) Value arrays: The recursion (40)–(41) computes the functions $\bar{v}_t : E \rightarrow \mathbb{R}$, $t = T, \dots, 0$. Each is an element of $\mathbb{R}^E$, which the bijection (43) identifies with $\mathbb{R}^\mathcal{K}$:

$$V_t := ((V_t)_k)_{k \in \mathcal{K}} \in \mathbb{R}^\mathcal{K}, \quad (V_t)_k := \bar{v}_t(g_k). \quad (44)$$

For an array $A \in \mathbb{R}^\mathcal{K}$ and a point $\tilde{y} \in \mathcal{X}$ whose cell index vector k is given by (16), let

$$\mathcal{I}(A, \tilde{y}) := \sum_{\kappa \in \{0, 1\}^d} \omega_\kappa(\tilde{y}) A_{k+\kappa}, \quad (45)$$

a weighted sum of the 2^d entries of A whose positions are obtained from k by adding 0 or 1 to each coordinate. Taking $A = V_{t+1}$ gives the interpolation (24) of $\bar{v}_{t+1}$ at $\tilde{y}$, since the entries of V_{t+1} are the values $\bar{v}_{t+1}(g_{k+\kappa})$ at the vertices of the cell by (44).

By (40), $\bar{v}_t$ is determined by $\bar{v}_{t+1}$ alone, and by no $\bar{v}_s$ with $s > t+1$. The whole family $V_0, \dots, V_T$ is nonetheless retained, since the policy (42) at time t involves $\bar{v}_{t+1}$ and is re-evaluated in Section III-C.

3) Noises and candidate controls: Fix once and for all an enumeration of the noise support,

$$\mathbb{W} = \{w_{(1)}, \dots, w_{(L)}\}, \quad L := |\mathbb{W}|, \quad (46)$$

each $w_{(\ell)} = (q_{(\ell)}, p_{(\ell)})$ being a pair of vectors of $\mathbb{R}^d$ as in Section II-A1: $q_{(\ell)} = (q_{(\ell)}^1, \dots, q_{(\ell)}^d)$ collects the inflows into dams $1, \dots, d$, and $p_{(\ell)} = (p_{(\ell)}^1, \dots, p_{(\ell)}^d)$ the prices at dams $1, \dots, d$, in the ℓ -th possible noise outcome. For $t = 0, \dots, T-1$ and $\ell = 1, \dots, L$, let

$$\rho_{t,\ell} := \mathbb{P}(W_t = w_{(\ell)}) \quad (47)$$

be the probability that the noise at time t takes its ℓ -th value, the letter ρ being used because p denotes prices. For each t , $(\rho_{t,1}, \dots, \rho_{t,L})$ is the law of W_t , so that $\sum_{\ell=1}^L \rho_{t,\ell} = 1$. These are the only probabilities the recursion uses: by (34),

$$\bar{v}_t(x) = \sum_{\ell=1}^L \rho_{t,\ell} v_t(x, w_{(\ell)}), \quad (48)$$

where $v_t(x, w_{(\ell)})$ is the minimum (39) at the noise $w_{(\ell)}$, so $\rho_{t,\ell}$ is the weight given to the ℓ -th outcome when averaging over the noise. The index t is needed because the noises are independent across t but not assumed identically distributed. For example, the inflows of a spring month do not need to follow the same law as those of a summer month.

Enumerate likewise the full product control grid,

$$\mathcal{U} := \mathcal{U}^1 \times \dots \times \mathcal{U}^d = \{u_{(1)}, \dots, u_{(m^d)}\}. \quad (49)$$

An element of $\mathcal{U}$ is called a *candidate control*: a grid control examined by the minimization (39) at some (t, x, w) , and admissible there if it satisfies (5), that is, if it belongs to $C_t(x, w)$. $\mathcal{U}$ is not the admissible set: by (15), $C_t(x, w) \subseteq \mathcal{U}$, with strict inclusion in general. But $\mathcal{U}$ depends on neither t , x nor w , so the same set of candidates is examined in each of the $T \times n^d \times L$ minimizations performed by the recursion, one per (t, x, w) . Algorithm 1 of Section III-B tests each candidate for admissibility, so that the minimization can run over $\mathcal{U}$.

Worked example: Consider a linear cascade of $d = 2$ dams, $\mathcal{P}(1) = \emptyset$ and $\mathcal{P}(2) = \{1\}$, with $[\underline{x}^i, \bar{x}^i] = [0, 100]$ and $[\underline{u}^i, \bar{u}^i] = [0, 40]$ for $i = 1, 2$, in arbitrary volume units, and $\varepsilon = 0$.

Grid. With $n = 5$, (12) gives $\delta^i = 25$ and $\mathcal{G}^i = \{0, 25, 50, 75, 100\}$, so that $g_{k_i}^i = 25 \times k_i$. The set $\mathcal{K} = \{0, \dots, 4\}^2$ has 25 elements; for instance $k = (2, 1)$ labels the grid point $g_k = (g_2^1, g_1^2) = (50, 25)$, dam 1 being half full and dam 2 at a quarter of its capacity. Each V_t is a 5×5 array, whose entry in row k_1 and column k_2 is $\bar{v}_t(25k_1, 25k_2)$.

Noises and controls. Let $L = 2$, with a dry outcome $w_{(1)}$ and a wet outcome $w_{(2)}$:

$$\begin{aligned} q_{(1)} &= (10, 5), & p_{(1)} &= (1, 1), \\ q_{(2)} &= (30, 15), & p_{(2)} &= (1, 1). \end{aligned}$$

In the dry outcome, $q_{(1)}^1 = 10$ units of water flow into dam 1 and $q_{(1)}^2 = 5$ into dam 2 during the month; in the wet outcome, 30 and 15. The price is 1 per unit of turbinated water at both dams in both outcomes, prices being deterministic in the

experiments ^[1]. Two months $t \neq t'$ may, for instance, have $\rho_{t,1} = \mathbb{P}(W_t = w_{(1)}) = 0.7$ and $\rho_{t,2} = \mathbb{P}(W_t = w_{(2)}) = 0.3$, while $\rho_{t',1} = 0.2$ and $\rho_{t',2} = 0.8$: the dry outcome is the more likely at t , and the less likely at t' . By (48), the value at t is then $\bar{v}_t(x) = 0.7 v_t(x, w_{(1)}) + 0.3 v_t(x, w_{(2)})$. The support $\mathbb{W}$ is common to all months; only the probabilities change. With $m = 3$, $\mathcal{U}^i = \{0, 20, 40\}$ and $\mathcal{U}$ has 9 elements. At each t , the recursion thus performs $|\mathcal{K}| \times L = 50$ minimizations, each over at most 9 candidates.

One transition. The recursion performs one minimization (39) for each grid point and each noise outcome; we follow one of them. Fix the grid point $x = g_k$ with $k = (2, 1)$, that is $x = (50, 25)$, and the wet outcome $w_{(2)}$: under the HD structure the month's inflows are known before the release is chosen, so the noise is fixed first. The minimization at $(x, w_{(1)})$ proceeds in the same way with the dry inflows, and (48) then averages the two. Consider the *candidate control* $u = (20, 40) \in \mathcal{U}$, that is, dam 1 releases 20 units and dam 2 releases 40. Dam 1 has no upstream dam, so $z^1 = 0$; (5) requires $u^1 \leq \min(40, 50 + 30 + 0 - 0) = 40$, which holds, and (2) gives $s^1 = \max(0, 50 - 20 + 30 + 0 - 100) = 0$. By (3), $z^2 = u^1 + s^1 = 20$; (5) requires $u^2 \leq \min(40, 25 + 15 + 20 - 0) = 40$, which holds, and $s^2 = \max(0, 25 - 40 + 15 + 20 - 100) = 0$. Therefore, $u = (20, 40)$ is admissible, and (4) gives the target point

$$\tilde{y} = (50 - 20 + 30, 25 - 40 + 15 + 20) = (60, 20).$$

This means that at the end of the month, dam 1 holds 60 units and dam 2 holds 20; neither spills with both volumes being below the capacity 100. Since 60 and 20 are not multiples of $\delta^i = 25$, $\tilde{y}$ is not a grid point.

Cell, vertices and weights. By (16), the cell index vector of $\tilde{y}$ is:

$$k = (\lfloor 60/25 \rfloor, \lfloor 20/25 \rfloor) = (2, 0),$$

and by (18), $(\lambda^1, \lambda^2) = (0.4, 0.8)$. The cell is $[50, 75] \times [0, 25]$, and its vertices and their weights (21) are:

κ	$k + \kappa$	$g_{k+\kappa}$	$\omega_\kappa(\tilde{y})$
(0, 0)	(2, 0)	(50, 0)	$0.6 \times 0.2 = 0.12$
(0, 1)	(2, 1)	(50, 25)	$0.6 \times 0.8 = 0.48$
(1, 0)	(3, 0)	(75, 0)	$0.4 \times 0.2 = 0.08$
(1, 1)	(3, 1)	(75, 25)	$0.4 \times 0.8 = 0.32$

The weights sum to one, as (22) states, and $0.12(50, 0) + 0.48(50, 25) + 0.08(75, 0) + 0.32(75, 25) = (60, 20) = \tilde{y}$, as (23) states.

Interpolated continuation value. Take $t = T - 1$, the last decision month. Then $V_{t+1} = V_T$, whose entries are, by (41) and (44), the terminal costs $(V_T)_k = \phi(g_k)$. With targets $\hat{x} = (75, 50)$ and weights $\alpha^1 = \alpha^2 = 0.01$ in (7),

$$\phi(x_T) = 0.01 \max(0, 75 - x^1)^2 + 0.01 \max(0, 50 - x^2)^2,$$

so that, at the four positions $k + \kappa$ of the table,

$$\begin{aligned} (V_T)_{(2,0)} &= \phi(50, 0) = 6.25 + 25 = 31.25, \\ (V_T)_{(2,1)} &= \phi(50, 25) = 6.25 + 6.25 = 12.5, \\ (V_T)_{(3,0)} &= \phi(75, 0) = 0 + 25 = 25, \\ (V_T)_{(3,1)} &= \phi(75, 25) = 0 + 6.25 = 6.25, \end{aligned}$$

more stored water meaning a smaller shortfall penalty. Then (45) gives:

$$0.12(31.25) + 0.48(12.5) + 0.08(25) + 0.32(6.25) = 13.75.$$

With the running cost (6), $c_t(x, u, w_{(2)}) = -1 \cdot 20 - 1 \cdot 40 = -60$, so the *candidate control* $u = (20, 40)$ receives the value $-60 + 13.75 = -46.25$ in the minimization (39) at $(x, w_{(2)})$. The other candidates of $\mathcal{U}$ are evaluated in the same way, and the smallest value is retained. At an earlier month $t < T - 1$, the entries of V_{t+1} are those produced by the previous step of the recursion, (40) at time $t + 1$, and enter (45) in the same way.

An inadmissible candidate. At the empty state $x = g_k$ with $k = (0, 0)$, that is $x = (0, 0)$, under the dry noise $w_{(1)}$, (5) gives $u^1 \leq \min(40, 0 + 10 + 0 - 0) = 10$, so $u^1 = 0$ is the only admissible grid value. Then $s^1 = 0$ and $z^2 = 0$, and $u^2 \leq \min(40, 0 + 5 + 0 - 0) = 5$, so $u^2 = 0$. Here $C_t(x, w_{(1)}) = \{(0, 0)\}$, a single element of the nine in $\mathcal{U}$.

4) Role of the time index: Two dependences on t must be distinguished. The arguments of the maps (2)–(4) and (6) depend on t : by the convention of footnote 1, x , u and w stand for realizations of X_t , U_t and W_t , which differ from one time step to the next. However, the maps themselves do not. The parameters entering them, $\underline{x}^i$, $\bar{x}^i$, $\underline{u}^i$, $\bar{u}^i$, ε and the valley topology $\mathcal{P}$, are constant in t , the prices being part of the noise. Hence $f_t(x, u, w) = f_{t'}(x, u, w)$ and $c_t(x, u, w) = c_{t'}(x, u, w)$ for all t, t' and all (x, u, w) . Along a trajectory, f_t and $f_{t'}$ are evaluated at the different realizations (X_t, U_t, W_t) and $(X_{t'}, U_{t'}, W_{t'})$, and return different values for that reason alone; evaluated at the same triple, they coincide. The subscript t is kept for consistency with the dynamics f_k of Definition 3.8 and the running costs r_k of equation (3.4)^[4], both of which may depend on time; the model instantiates them with time-invariant maps. The time dependence of the problem is thus carried entirely by the law $(\rho_{t,\ell})_\ell$ of W_t and by the array V_{t+1} entering (40).

B. Backward induction

This subsection turns (39)–(42) into two algorithms, written with the Bellman operators of Section 3.3^[4]. The first evaluates, for one candidate control, the quantity minimized in (39); the second applies (40)–(41) at every grid point, from $t = T - 1$ to $t = 0$. Minimization replaces the maximization used in^[4], as Definition 3.10^[4] allows.

1) Bellman operators: For the augmented Markov decision process of Section II-D, a time t , a function $v \in \mathbb{R}^{E \times \mathbb{W}}$, a pair $(x, w) \in E \times \mathbb{W}$ and a control $u \in C_t(x, w)$, let

$$\begin{aligned} [\mathcal{B}^{(t,u)}(v)](x, w) &:= c_t(x, u, w) \\ &+ \sum_{(y, w') \in E \times \mathbb{W}} M_{(x,w),(y,w')}^{(t,u)} v(y, w'). \end{aligned} \quad (50)$$

This is the value at (x, w) of the Bellman operator with frozen policy $\mathcal{B}^{(t,\pi)}$, written $B^{(t,\pi)}$ in Definition 3.15^[4], for any feedback policy π at time t with $\pi(x, w) = u$: as noted there,

Algorithm 1 Evaluation of $[\mathcal{B}^{(t,u)}(v_{t+1})](x, w)$

```

1: function EVAL( $x, w, u, V_{t+1}$ )  $\triangleright w = (q, p)$ 
2:   for  $i = 1, \dots, d$  do
3:      $z^i \leftarrow \sum_{j \in \mathcal{P}(i)} (u^j + s^j)$ 
4:     if  $u^i > x^i + q^i + z^i - \underline{x}^i$  then
5:       return  $+\infty$ 
6:     end if
7:      $s^i \leftarrow \max(0, x^i - u^i + q^i + z^i - \bar{x}^i)$ 
8:      $\tilde{y}^i \leftarrow x^i - u^i + q^i + z^i - s^i$ 
9:   end for
10:   $c \leftarrow \sum_{i=1}^d (-p^i u^i + \varepsilon (u^i)^2)$ 
11:  return  $c + \mathcal{I}(V_{t+1}, \tilde{y})$ 
12: end function

```

that value depends on π only through $\pi(x, w)$. The Bellman operator at time t , equation (3.6) of^[4], is then

$$[\mathcal{B}^{(t)}(v)](x, w) = \min_{u \in C_t(x, w)} [\mathcal{B}^{(t,u)}(v)](x, w), \quad (51)$$

and (32) reads $v_t = \mathcal{B}^{(t)}(v_{t+1})$, as in (3.7) of^[4]. For $v = v_{t+1}$, the reduction (35)–(38) and (45) give

$$[\mathcal{B}^{(t,u)}(v_{t+1})](x, w) = c_t(x, u, w) + \mathcal{I}(V_{t+1}, \tilde{y}), \quad (52)$$

where $\tilde{y} = f_t(x, u, w)$. The right-hand side depends on v_{t+1} only through the array V_{t+1} , and is well defined for every $x \in \mathcal{X}$, not only for $x \in E$; Section III-C uses it at such points. In this notation, (39) and (48) become:

$$(V_t)_k = \sum_{\ell=1}^L \rho_{t,\ell} [\mathcal{B}^{(t)}(v_{t+1})](g_k, w_{(\ell)}), \quad k \in \mathcal{K}, \quad (53)$$

and the policy (42) is:

$$\pi_t(x, w) \in \arg \min_{u \in C_t(x, w)} \left\{ c_t(x, u, w) + \mathcal{I}(V_{t+1}, f_t(x, u, w)) \right\}.$$

2) Evaluating one candidate: In the algorithms, $a \leftarrow b$ assigns the value b to a . Algorithm 1 evaluates (52) for a given triple $(x, w, u) \in \mathcal{X} \times \mathbb{W} \times \mathcal{U}$, and returns $+\infty$ when $u \notin C_t(x, w)$. Since $\underline{u}^i = 0$ and $u \in \mathcal{U}$, every candidate satisfies $\underline{u}^i \leq u^i \leq \bar{u}^i$, and admissibility reduces to the second bound of (5), $u^i \leq x^i + q^i + z_t^i - \underline{x}^i$ for $i = 1, \dots, d$. The dams are processed in increasing index: when z_t^i is formed at line 3, every s_t^j with $j \in \mathcal{P}(i)$ is already available, since $j < i$. For an inadmissible candidate the algorithm returns at line 5, before the cell index (16) is formed inside $\mathcal{I}$, so that (16) is only evaluated at points of $\mathcal{X}$, as assumed in Section II-C.

Since $+\infty$ exceeds every real value, an inadmissible candidate never attains a minimum over $\mathcal{U}$ as long as an admissible one exists. Assigning $+\infty$ to inadmissible candidates therefore allows the minimization over $C_t(x, w)$ in (51) to be written over $\mathcal{U}$:

$$\min_{u \in \mathcal{U}} \text{EVAL}(x, w, u, V_{t+1}) = [\mathcal{B}^{(t)}(v_{t+1})](x, w), \quad (54)$$

the right-hand side being finite since $C_t(x, w) \neq \emptyset$ (Section II-D).

Algorithm 2 Backward induction

```

1: for  $k \in \mathcal{K}$  do
2:    $(V_T)_k \leftarrow \phi(g_k)$ 
3: end for
4: for  $t = T - 1, \dots, 0$  do
5:   for  $k \in \mathcal{K}$  do
6:     for  $\ell = 1, \dots, L$  do
7:        $h_\ell \leftarrow \min_{u \in \mathcal{U}} \text{EVAL}(g_k, w_{(\ell)}, u, V_{t+1})$ 
8:     end for
9:      $(V_t)_k \leftarrow \sum_{\ell=1}^L \rho_{t,\ell} h_\ell$ 
10:   end for
11: end for
12: return  $V_0, \dots, V_T$ 

```

3) Backward recursion: Algorithm 2 applies (53) at every grid point, for $t = T - 1, \dots, 0$, starting from (41). The arrays it fills are exactly those of (44), by backward induction on t . At $t = T$, line 2 sets each entry to $\phi(g_k) = \bar{v}_T(g_k)$ by (41). If the array held for $t + 1$ is V_{t+1} , then line 7 computes $[\mathcal{B}^{(t)}(v_{t+1})](g_k, w_{(\ell)})$ by (54) and (52), which is $v_t(g_k, w_{(\ell)})$ by (39), and line 9 forms their average (48), which is $\bar{v}_t(g_k) = (V_t)_k$. The arrays are therefore exact for the discretized problem; the discretization of Section II is the only approximation made. The value of the problem is $(V_0)_k$ if $x_0 = g_k$, and otherwise the interpolation of V_0 at x_0 given in Section II-D.

Only minimal values are retained at line 7, not the minimizing controls: the policy (42) is not stored, and Section III-C recovers it from the arrays V_{t+1} when needed, with the same function EVAL.

C. Simulation

Algorithm 2 solves the discretized Markov decision process of Section II-D exactly, so that $\bar{v}_0(x_0)$ is the value of that process. It is not the value of (9):

$$\min_{\sigma} J^{(T, \sigma)}.$$

The two differ in their admissible sets, $C_t(x, w) \subseteq C_t^c(x, w)$ by (15), and in their dynamics: given (x, u, w) , (4) produces the single point $f_t(x, u, w) \in \mathcal{X}$, whereas the volume component of the discretized process is the random variable $Y_{t+1} = G(f_t(x, u, w), \xi_{t+1})$ of (28), which takes its values in E and is distributed over the 2^d vertices of the cell containing $f_t(x, u, w)$ according to (26). Thus $\bar{v}_0(x_0)$ is the exact value of a problem other than (9), and no inequality between the two is established here.

What Algorithm 2 does produce for (9) is a strategy. This subsection evaluates it by Monte Carlo simulation of the dynamics (4), as in the forward pass of^[2] and the simulation stage of^[11].

1) The induced strategy: For $(x, w) \in \mathcal{X} \times \mathbb{W}$ and $t \leq T - 1$, let

$$\pi_t(x, w) \in \arg \min_{u \in C_t(x, w)} \left\{ c_t(x, u, w) + \mathcal{I}(V_{t+1}, f_t(x, u, w)) \right\}, \quad (55)$$

the right-hand side being that of (52), which is defined for every $x \in \mathcal{X}$ and depends on v_{t+1} only through the array V_{t+1} returned by Algorithm 2. The set $C_t(x, w)$ is finite and nonempty for every $(x, w) \in \mathcal{X} \times \mathbb{W}$ (Section II-D), so the minimum in (55) is attained. Its set of minimizers need not be a singleton: distinct candidates may receive equal values in (52). Any selection from that set is an optimal strategy of the discretized problem, by Theorem 3.13^[4], but two minimizers may send the state to different points of $\mathcal{X}$ under (4), hence generate different trajectories and different values of $J^{(T, \pi)}$. One selection is therefore fixed: $\pi_t(x, w)$ is the minimizer of smallest index in the enumeration (49). The rule is arbitrary; what matters is only that it be the same at every evaluation, so that π_t is a function and $J^{(T, \pi)}$ a single number.

Since $C_t(x, w) \subseteq C_t^c(x, w)$ by (15), and since $\pi_t(x, w)$ depends only on the state and the noise at time t , the sequence $\pi = (\pi_t)_{0 \leq t \leq T-1}$ is an admissible feedback strategy for (9) in the sense of Definitions 3.2 and 3.4^[4]. It is optimal for the discretized process, not for (9); being admissible, it satisfies:

$$\min_{\sigma} J^{(T, \sigma)} \leq J^{(T, \pi)}, \quad (56)$$

so that $J^{(T, \pi)}$ is an upper bound on the value of (9).

2) Scenarios and trajectories: A *scenario* is one realization of the whole noise sequence. The simulation uses N of them, indexed by $r = 1, \dots, N$:

$$w^{(r)} = (w_0^{(r)}, \dots, w_{T-1}^{(r)}) \in \mathbb{W}^T, \quad (57)$$

the T entries of a scenario being drawn independently, $w_t^{(r)}$ from the law $(\rho_{t, \ell})_{\ell} \in W_t$ given by (47), and the N scenarios independently of one another. Each $w_t^{(r)}$ is thus one of the L elements $w_{(1)}, \dots, w_{(L)}$ enumerated in (46). The support $\mathbb{W}$ is the one of Section II-A1: the inflows of (9) are already discrete, so the simulation removes the discretization of the state space and of the controls, and nothing else.

Along the scenario r , the trajectory is generated by (55) and (4),

$$\begin{aligned} X_0 &= x_0, \\ U_t &= \pi_t(X_t, w_t^{(r)}), \\ X_{t+1} &= f_t(X_t, U_t, w_t^{(r)}), \end{aligned} \quad (58)$$

and the realized cost of (8) along it is

$$Z^{(r)} = \sum_{t=0}^{T-1} c_t(X_t, U_t, w_t^{(r)}) + \phi(X_T). \quad (59)$$

The interpolation of Section II-C enters (58) only inside (55), to select the release. It does not act on the state: by (4), X_{t+1} is the exact next volume, so the trajectory leaves E at the first step and thereafter visits points of $\mathcal{X}$ that are not grid points.

3) Estimator: The $Z^{(r)}$, $r = 1, \dots, N$, are independent and identically distributed, with common mean $J^{(T, \pi)}$ by (8) and (59). The empirical mean and variance,

$$\begin{aligned} \hat{J}_N &= \frac{1}{N} \sum_{r=1}^N Z^{(r)}, \\ \hat{\varsigma}_N^2 &= \frac{1}{N-1} \sum_{r=1}^N (Z^{(r)} - \hat{J}_N)^2, \end{aligned} \quad (60)$$

Algorithm 3 Simulation of the induced strategy

```

1: for  $r = 1, \dots, N$  do
2:    $X \leftarrow x_0, \quad Z^{(r)} \leftarrow 0$ 
3:   for  $t = 0, \dots, T-1$  do
4:     draw  $w$  from  $(\rho_{t, \ell})_{\ell}$ 
5:      $U \leftarrow \arg \min_{u \in \mathcal{U}} \text{EVAL}(X, w, u, V_{t+1})$ 
6:      $Z^{(r)} \leftarrow Z^{(r)} + c_t(X, U, w)$ 
7:      $X \leftarrow f_t(X, U, w)$ 
8:   end for
9:    $Z^{(r)} \leftarrow Z^{(r)} + \phi(X)$ 
10: end for
11: return  $\hat{J}_N$  and  $\hat{\varsigma}_N$  as in (60)

```

are therefore unbiased for $J^{(T, \pi)}$ and for the variance of $Z^{(1)}$. The costs being bounded, that variance is finite, and the central limit theorem gives the asymptotic confidence interval at level 95%:

$$\left[\hat{J}_N - 1.96 \hat{\varsigma}_N / \sqrt{N}, \quad \hat{J}_N + 1.96 \hat{\varsigma}_N / \sqrt{N} \right], \quad (61)$$

which is the criterion used in^[2] to assess the accuracy of a forward pass. Only $\hat{J}_N$ is comparable to the values reported in^[1], those being obtained in the same way.

4) The algorithm: Algorithm 3 implements (57)–(60). It calls the function EVAL of Algorithm 1, which returns $+\infty$ at inadmissible candidates, so that line 5 is (55) written over $\mathcal{U}$, exactly as (54) is (51). The arrays $V_1, \dots, V_T$ are those returned by Algorithm 2, and are not recomputed: the backward recursion is run once, and the N trajectories differ only through the noise drawn at line 4. It is at line 7 that the two models part, the state being advanced by the dynamics (4) rather than scattered onto a vertex.

One remark. When several grid resolutions n are compared in Section IV, the same N scenarios are used for all of them. The differences between resolutions are then differences between policies evaluated on common noise, and are not contaminated by the sampling error (61), which for moderate N may be of the same order.

D. Complexity

Algorithm 2 calls EVAL once per quadruple $(t, k, \ell, u) \in \{0, \dots, T-1\} \times \mathcal{K} \times \{1, \dots, L\} \times \mathcal{U}$, that is $T n^d L m^d$ times. One such call costs $\Theta(d 2^d)$ arithmetic operations: the loop over the dams is $\Theta(d)$, each dam belonging to at most one set $\mathcal{P}(j)$, and so is the running cost at line 10, while line 11 evaluates (45), a sum of 2^d terms whose weights are the products (21) of d factors each. The rest of the recursion: the n^d evaluations of ϕ , the comparisons realizing the minima at line 7 and the averages at line 9, does not carry that factor and is dominated. The backward pass therefore costs:

$$\Theta(T L d (2nm)^d), \quad (62)$$

having grouped $n^d m^d 2^d = (2nm)^d$. The arrays it returns hold $(T+1) n^d$ real numbers by (44), which is the whole of what passes to Algorithm 3; that algorithm costs $\Theta(N T d (2m)^d)$, in which n does not appear, a trajectory never visiting the grid exhaustively. The simulation is thus negligible beside (62) as

soon as $N \ll L n^d$, and the limits of the method are set by the backward pass alone.

Three factors of (62) are exponential in the number of dams, with distinct origins. The factor n^d is $|E|$: the state is one volume per dam, so the grid (14) is a product of d grids. The factor m^d is $|\mathcal{U}|$, for the same reason applied to the releases. These are not choices as both follow from the valley having d dams and from the decision to discretize. The factor 2^d is the number of vertices of a cell, and is the price of the interpolation rule of Section II-C. Note that the nearest-neighbour rounding sends each transition to a single grid point and would give $\Theta(T L d (nm)^d)$ instead, a saving of 2^d against the bias discussed in Section II-C5. The exponential in d survives either way, and only T and L enter linearly. With $T = 12$, $L = 10$, $m = 5$ and $n = 20$, taken for illustration rather than from the instance of Section IV-A, (62) is of the order of 10^7 operations at $d = 2$ and of 5×10^{16} at $d = 6$: this is the gap that the decomposition of^{[1], [3]} and the cutting-plane approximation of^[2] are built to close, and the reason the present approach is confined to small valleys.

IV. NUMERICAL RESULTS

THIS section reports what Algorithms 2 and 3 produce on three valleys, of $d = 2, 4$ and 6 dams.⁶ Section IV-A fixes the instance and Section IV-B the quantities reported. Four experiments follow, each settling a claim left open earlier: Section IV-C compares the interpolation rule of Section II-C against nearest-neighbour rounding, as announced in Section II-C5; Section IV-D measures the cost (62) of the backward pass against the number of dams; Section IV-E measures the difference between $\bar{v}_0(x_0)$ and $J^{(T, \pi)}$, which Section III-C established to be values of two different problems without relating them; and Section IV-F isolates the part of that difference attributable to d alone. Section IV-G then reports two features of the solution with informative figures, and Section IV-H compares the results with those found in Carpentier et al^[1].

A. The instance

Carpentier et al.^[1] describe their academic valleys only qualitatively: no capacities, inflow laws, prices or targets are reported. However, they do state that all dams have roughly the same maximal volume, that turbine capacity increases and mean inflow decreases with depth in the valley, that the inflow laws are discrete with finite support, and that prices are deterministic. The numbers in Table I are ours and meet each of those statements. The four- and six-dam topologies of Fig. 1 follow their Fig. 4^[1]; the two-dam linear cascade is our assumption, their collection beginning at four dams.

Prices. The value p^i of one unit of turbinated volume at dam i is the market price of electricity times the energy produced per unit volume, and the latter is proportional to the head of

⁶Python 3.9.6 with NumPy 2.0.2 on an Apple M4 Pro (10 performance and 4 efficiency cores, 24 GB), macOS 26.6.2, the backward pass distributed over 14 worker processes. Carpentier et al.^[1] report their timings on a 3.4 GHz four-core Intel Xeon E3, therefore absolute running times are not comparable between the two papers; the ratios of Section IV-D are.

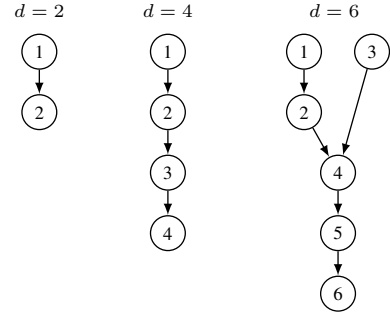


Fig. 1. The 2-dam, 4-dam and 6-dam valleys. An arrow from j to i means $j \in \mathcal{P}(i)$.

TABLE I
INSTANCE PARAMETERS.

Common to the three valleys	
horizon	$T = 12$ months
volumes	$\underline{x}^i = 0, \quad \bar{x}^i = 100$
target and initial	$\hat{x}^i = x_0^i = 50$
terminal weight	$\alpha^i = 0.1$
turbine cost	$\varepsilon = 10^{-3}$
noise support	$L = 5, \quad \nu = (0.4, 0.7, 1.0, 1.4, 2.0)$
turbine capacity	$\bar{u}^i = \text{round}(2 C^i), \quad \underline{u}^i = 0$
Per dam	
$d = 2$	
μ	(8.0, 3.0)
p	(1.3, 0.7)
$\bar{u}$	(16, 22)
$d = 4$	
μ	(8.0, 6.33, 4.67, 3.0)
p	(1.3, 1.1, 0.9, 0.7)
$\bar{u}$	(16, 29, 38, 44)
$d = 6$	
μ	(8.0, 6.75, 8.0, 5.5, 4.25, 3.0)
p	(1.3, 1.15, 1.3, 1.0, 0.85, 0.7)
$\bar{u}$	(16, 30, 16, 56, 65, 71)

dam i . Heads are larger in the steep upper reaches of a valley than on its flatter lower course, so p^i is taken to decrease with the depth of the dam, from 1.3 at a source to 0.7 at the outlet, normalised to approximately 1 on average. Prices do not depend on t : by (46) they are a component of w , and the support $\mathbb{W}$ is common to all months (Section III-A), so a seasonal price would require $\mathbb{W}$ to depend on t .

Inflows. Let μ^i denote the mean *environmental* inflow of dam i , that is the mean of Q_t^i alone: the rain, snowmelt and tributaries entering dam i directly. It does not include the water arriving from upstream, which enters the dynamics separately through z_t^i in (3). The inflow component of the ℓ -th outcome of $\mathbb{W}$ in (46) is then:

$$q_{(\ell)}^i = \nu_{\ell} \mu^i, \quad \nu = (0.4, 0.7, 1.0, 1.4, 2.0), \quad (63)$$

a single multiplier scaling every dam at once: the valley is wet or dry as a whole, the inflows being perfectly correlated across dams. The five values are ours. They are centered on the unit multiplier and asymmetric about it: the wettest outcome stands at twice the mean and the driest at four tenths of it. This is because river inflows are bounded below by zero and not above; $L = 5$ is the smallest support admitting a normal outcome, a dry and a wet one, and a tail on each side.

Table I takes μ^i to decrease with depth, from 8.0 at a

source to 3.0 at the outlet, as^[1] describes. This concerns the environmental component only. A headwater dam drains a large mountain catchment⁷ of its own, whereas a dam on the flatter lower course has a small *incremental* catchment, most of the water reaching it having already passed through the dams above. The total mean inflow of dam i , written C^i below, therefore rises downstream even as μ^i falls: in the six-dam valley μ runs from 8.0 to 3.0 while C runs from 8.0 to 35.5.

Seasonality. The inflows are seasonal, and by the constraint of the previous paragraph the seasonality is carried by the law $(\rho_{t,\ell})_\ell$ of (47) rather than by the inflow values. The law $\rho_{t,\ell}$ interpolates between a dry law and a wet law over the twelve months and peaks in spring, carrying the mean multiplier $\sum_\ell \rho_{t,\ell} \nu_\ell$ from 0.74 in November to 1.34 in May. Carpentier et al.^[1] report no inflow data for their academic valleys, and no conclusion below depends on the shape of the profile.

Turbine capacity. The maximal release $\bar{u}^i$ cannot be chosen independently of the valley topology. Dam i must pass the whole discharge of the dams above it, which by (3) carries their catchments as well as its own; write C^i for the mean inflow of the sub-valley above and including dam i . If $\bar{u}^i < C^i$, the volume climbs to $\bar{x}^i$ and (2) spills the excess at every step, after which releasing the maximum is optimal at every state and the problem ceases to be a dynamic one. We therefore set $\bar{u}^i = \gamma C^i$, rounded to the nearest unit, with $\gamma > 1$ leaving room to draw the reservoir down.

The constraint $u^i \leq \bar{u}^i$ of (5) is said to be *active*, or *binding*, at a triple (t, x, w) when the control selected by (55) has i -th component equal to $\bar{u}^i$: the turbine limit, rather than the comparison of costs, is then what fixes the release. We choose γ so that this occurs at times but not usually. Over the $N \times T \times d$ releases of the simulation at $d = 4$, the constraint is active in 98% of them when $\gamma = 1$, in 18% when $\gamma = 2$, and in 1% when $\gamma = 3$, at which point it has ceased to constrain anything. We take $\gamma = 2$.

B. Quantities reported

Two numbers are compared repeatedly below and must not be confused. $\bar{v}_0(x_0)$ is the value of the discretized Markov decision process, which Algorithm 2 computes exactly; $\hat{J}_N$ is the estimator (60) of the cost $J^{(T,\pi)}$ of the induced strategy under the exact dynamics (4). The first is what the discretized model predicts, the second what its strategy achieves. We call their difference the *gap*,

$$\text{gap} := \bar{v}_0(x_0) - \hat{J}_N, \quad (64)$$

and $\text{gap}/|\hat{J}_N|$ the *relative gap*, the two valleys of different size being otherwise not comparable. A positive gap means the discretized model predicts a higher cost than its own strategy incurs: the model *under-promises*, and $\bar{v}_0(x_0)$ is a conservative estimate of what the strategy achieves. A negative gap means it predicts a lower cost than its strategy incurs, so the model *over-promises*, announcing a revenue the strategy does not deliver.

⁷The *catchment* of a point on a river is the area of land from which rain and snowmelt drain into it. The catchment of dam i alone supplies Q_t^i ; the land draining into the dams above it supplies water that reaches dam i through z_t^i instead.

TABLE II

INTERPOLATION AGAINST NEAREST-NEIGHBOUR ROUNDING, $d = 2$, $m = 10$, $N = 10^4$. ERROR IS $|\hat{J}_N - \hat{J}_N^{\text{ref}}|$ AGAINST THE REFERENCE AT $n = 1281$; $\pm$ ARE PAIRED HALF-WIDTHS.

n	δ	interpolation		rounding	
		error	$\pm$	error	$\pm$
6	20.000	2.976	0.006	74.474	0.965
11	10.000	0.529	0.004	28.903	0.409
21	5.000	0.0247	0.002	5.053	0.124
41	2.500	0.0065	0.001	1.525	0.027
81	1.250	0.0088	0.002	0.976	0.015
161	0.625	0.0018	0.001	0.796	0.008
321	0.313	0.0007	0.0006	0.531	0.005
641	0.156	0.00004	0.0003	0.222	0.006

TABLE III

GAP (64) UNDER THE TWO RULES, SAME RUNS AS TABLE II. THE RULE MARKS WHERE THE INTERPOLATION GAP FALLS BELOW ITS OWN HALF-WIDTH OF 0.56: THE ENTRIES BENEATH IT ARE NOT RESOLVED AT $N = 10^4$ AND ARE RE-MEASURED IN TABLE V.

n	δ	interpolation	rounding
6	20.000	+3.75	-240.24
11	10.000	+1.46	-141.83
21	5.000	+0.60	-58.22
41	2.500	+0.28	-27.79
81	1.250	+0.14	-13.64
161	0.625	+0.11	-7.23
321	0.313	+0.08	-3.61
641	0.156	+0.07	-1.76

The *half-width* is $1.96 \hat{\epsilon}_N / \sqrt{N}$, half the length of the confidence interval (61). It is the random error carried by $\hat{J}_N$, and hence by every quantity computed from it: the gap primarily, since $\bar{v}_0(x_0)$ is exact and contributes none, but also the error columns of Table II and the differences between rows used below. A gap larger than its own half-width is *resolved*: drawing the N scenarios again moves it by about one half-width, which leaves its size and its sign intact, so it may be read and compared across rows. A gap smaller than its own half-width is not, its confidence interval containing zero: the measurement cannot separate it from no effect at all, and the figure is then quoted as an upper bound on the true value. The interpolation gap of 0.28 at $n = 41$ in Table III is of the second kind, its half-width being 0.56; the rounding gap of -27.79 on the same grid is of the first.

Finally, a difference between two rows of a table is *paired* when both are simulated on the same N scenarios, as described at the end of Section III-C. The two rows then carry the same sampling error, which cancels in the difference, so a paired difference has a half-width of its own, smaller than that of either row and reported alongside it. A difference too small to read against the rows it comes from may still be resolved when paired.

C. Interpolation against rounding

Section II-C5 claimed that the unbiasedness (23) of multilinear interpolation buys accuracy on a coarser grid, and quoted the resolutions measured here. Table II is that measurement: the two-dam valley with $m = 10$ and the same $N = 10^4$ scenarios in every row, each rule compared against a reference solved at $n = 1281$.

The claim holds. Interpolation at $n = 11$ attains an error of 0.529, which rounding does not match until $n = 321$, where its

error is 0.531: the factor of 29 in the resolution, and hence of 852 in $|E|$ and in the cost (62), reported in Section II-C5. The two columns moreover converge at the rates the bias argument predicts: halving δ divides the interpolation error by roughly four and the rounding error by roughly two, second order against first. The advantage therefore widens as the grid is refined.

Two limits bound how far the table may be read. The interpolation column is not monotone, the minimization (39) being over the finite set $\mathcal{U}$, so a finer grid changes the selected release in jumps and the induced strategy need not improve at every step; Section IV-E returns to the matter. The last row, in turn, compares two neighbouring grids, which share most of their error, so 0.00004 understates the true error at $n = 641$.

The two rules differ in one further respect, which the accuracy columns do not show: the sign of the gap (64). Table III shows the two columns never crossing zero: interpolation under-promises on every grid, rounding over-promises on every grid. On the coarsest grid the difference is extremely clear: the rounding model announces a cost of -399.8 while its strategy realizes -159.5 , two and a half times the revenue it delivers, where interpolation announces -227.3 and delivers -231.0 . Only one of the two rules fails safely, which is a second reason to prefer it. The rounding column is resolved throughout; the interpolation column is not, from $n = 41$ down, and read again at $N = 10^6$ in Table V the two-dam gap does not level off near 0.08 but continues to 0.009.

D. Cost against the number of dams

Table IV measures the growth (62) of the backward pass in two ways. Panel (a) holds the grid fixed at a rudimentary configuration and adds dams; it runs on one core, so that nothing but the problem size varies. Panel (b) asks how finely each dam's volume axis can be cut: n points per dam, or equivalently a spacing δ , a grid being finer when n is larger and δ smaller. Each row is the finest grid this study solved for that valley.

TABLE IV
COST OF ALGORITHM 2. (A) FIXED RESOLUTION, SINGLE PROCESS; (B) FINEST RESOLUTION AFFORDABLE, 14 PROCESSES. ONE WORK UNIT IS ONE CALL TO EVAL.

(a) $n = 5, m = 3$					
d	time		ratio	units/s	
2	6.2 ms		—	1.3×10^7	
4	0.83 s		133	7.4×10^7	
6	14.3 min		1035	5.6×10^7	
(b) finest affordable					
d	n	m	δ	time	units/s
2	1281	10	0.08	84 s	7.0×10^8
4	16	10	6.67	27.9 min	4.7×10^8
6	7	4	16.67	110.5 min	3.1×10^8

Adding two dams at a fixed grid costs three orders of magnitude, and neither step matches the growth in work units that (62) predicts, 750 from $d = 2$ to $d = 4$ and 788 from $d = 4$ to $d = 6$. The two miss in opposite directions. At $d = 2$ the pass is too small to reach full speed, most of its 6 ms going on allocation rather than arithmetic, so the measured ratio of 133 understates the growth; at $d = 6$ the interpolation (45)

gathers $2^d = 64$ scattered entries of V_{t+1} for every candidate examined, which costs more per unit of work than at $d = 4$, so the measured 1035 overstates it. (62) counts arithmetic, and arithmetic is not the whole cost.

Panel (b) puts the same growth in the units a modeller cares about. The affordable product nm falls from 12,810 at two dams to 28 at six and the mesh coarsens from $\delta = 0.08$ to 16.67: seventy-nine times the waiting for a state description two hundred times coarser. Resolving six dams as finely as two would take some 8×10^9 years at the measured rate, and merely matching the four-dam mesh eleven days. The simulation, by contrast, is the negligible part Section III-D claimed it would be whenever $N \ll L n^d$, a few percent of the total at these grids, but its hypothesis fails precisely where accuracy binds: resolving the two-dam gap took $N = 10^6$ against $L n^d = 5.2 \times 10^5$, and 42 minutes of that 4.4 hour run went to Algorithm 3.

E. The discretized value against the realized cost

Section III-C established that $\bar{v}_0(x_0)$ is the exact value of a process other than (9) and declined to assert any inequality between them. Table V measures the difference at four meshes per valley.

The difference has a definite sign: the gap is positive at every mesh and every d , as the interpolation column of Table III indicated, so the discretized model under-predicts the performance of its own strategy. It falls as the mesh is refined, at a rate read from the paired *drop* column, which is resolved even where the gap itself is not: faster than δ and slower than δ^2 in all three valleys, which is as much as four points per sequence can establish. At $d = 2$, $\delta = 0.31$ the gap reaches $+0.009$ against a half-width of 0.056, indistinguishable from zero. The gap is therefore not a defect of the construction of Section II-D but the price of a coarse grid.

Refinement is not monotone, however. At six dams the gap falls to 25.834, rises to 26.601 and falls again to 20.599, and the rise of 0.767 against a paired half-width of 0.145 is real. Splitting each drop into the movement of $\bar{v}_0(x_0)$, which is deterministic, and that of $\hat{J}_N$ locates the cause: from $n = 4$ to $n = 5$ the realized cost of the strategy worsens by 5.910 ± 0.113 while $\bar{v}_0(x_0)$ improves. A finer grid is thus not guaranteed to induce a better strategy. With $m = 4$ release levels per dam, a better $\bar{v}_{t+1}$ flips the argmin of (55) at some states, and a flip is discontinuous – the mechanism already met in Section IV-C.

The two value columns separate two kinds of convergence. Across the four-dam sequence $\hat{J}_N$ settles early, moving by only 0.84 (0.11%) from $\delta = 14.29$ onwards while $\bar{v}_0(x_0)$ still moves by 5.08; at six dams the totals are 15.4 for $\hat{J}_N$ against 50.7 for $\bar{v}_0(x_0)$. The strategy therefore converges substantially faster than its valuation: a coarse grid yields a nearly optimal policy together with an unreliable estimate of what that policy is worth. Since $\bar{v}_0(x_0)$ is still moving by more than 10 at the finest six-dam mesh affordable, the value of that valley is reported here as at most -1538 , not as -1538 .

F. The effect of the number of dams

Read across the blocks of Table V, the relative gap at $\delta = 25$ is 1.06% at $d = 4$ and 1.68% at $d = 6$, which could mean

TABLE V

THE GAP (64) AT FOUR MESHES PER VALLEY. $N = 10^6$ AT $d = 2$, 10^4 AT $d = 4$, 4000 AT $d = 6$. THE *drop* IS THE DECREASE OF THE GAP SINCE THE ROW ABOVE AND IS PAIRED; ITS HALF-WIDTH IS ACCORDINGLY FAR BELOW THAT OF THE GAP ITSELF.

d	n	m	δ	$\bar{v}_0(x_0)$	$\hat{J}_N$	half-width	gap (rel.)	drop	$\pm$
2	41	10	2.500	-233.705	-233.912	0.056	+0.206 (0.09%)		
2	81	10	1.250	-233.845	-233.911	0.056	+0.066 (0.03%)	+0.141	0.000
2	161	10	0.625	-233.878	-233.917	0.056	+0.039 (0.02%)	+0.027	0.000
2	321	10	0.313	-233.908	-233.917	0.056	+0.009 (0.00%)	+0.029	0.000
4	5	10	25.00	-746.51	-754.51	1.807	+8.001 (1.06%)		
4	8	10	14.29	-768.78	-774.58	1.781	+5.799 (0.75%)	+2.202	0.095
4	12	10	9.09	-772.81	-774.99	1.783	+2.179 (0.28%)	+3.620	0.019
4	16	10	6.67	-773.86	-775.42	1.776	+1.556 (0.20%)	+0.623	0.020
6	4	4	33.33	-1487.40	-1543.25	5.524	+55.857 (3.62%)		
6	5	4	25.00	-1511.51	-1537.34	5.531	+25.834 (1.68%)	+30.023	0.113
6	6	4	20.00	-1527.72	-1554.32	5.537	+26.601 (1.71%)	-0.767	0.145
6	7	4	16.67	-1538.07	-1558.67	5.544	+20.599 (1.32%)	+6.002	0.180

TABLE VI

THE GAP AT FIXED MESH AND FIXED CONTROL GRID: $n = 5$, $m = 4$, $\delta = 25$, $N = 10^4$ IN EVERY ROW.

d	$\bar{v}_0(x_0)$	$\hat{J}_N$	gap	$\pm$	relative
2	-218.279	-224.293	+6.014	0.568	2.68%
4	-740.400	-751.523	+11.123	1.770	1.48%
6	-1511.509	-1532.240	+20.731	3.486	1.35%

that the gap grows with the number of dams. Those two rows also differ in m , ten release levels per dam against four, so the ratio of 1.6 confounds the dimension with the coarseness of the control grid. Table VI removes the confound: $n = 5$ and $m = 4$ throughout, so that $\delta = 25$ and $|\mathcal{U}^i|$ are identical and only d varies.

All three gaps are resolved at better than 5σ , and the relative gap *falls* with d : the fall is resolved from two to four dams and flat from four to six. The apparent growth in Table V is the control grid. Comparing the $d = 4$ row here against the $m = 10$ row of Table V at the same mesh, coarsening $\mathcal{U}$ alone multiplies the relative gap by 1.40 ± 0.38 , while the two extra dams, at fixed m , multiply it by 0.91 ± 0.21 . Neither factor is resolved on its own, but the second carries the opposite sign to the one Table V suggests.

The same gap can be divided three ways, and they disagree. Across Table VI the gap itself spans a factor 3.4 and the gap relative to $|\hat{J}_N|$ a factor 2.0, while the gap divided by the number of dams – 3.01 ± 0.28 , 2.78 ± 0.44 and 3.46 ± 0.58 – is constant within its error bars. Only the last is stable, because each reservoir contributes approximately the same absolute interpolation bias and these add, whereas the value of the valley grows faster than their sum – by 6.8 against 3.4 from two dams to six – since water released upstream is sold again at every dam below. The gap is therefore governed by δ and by the coarseness of $\mathcal{U}$, and grows with d only in the sense that a valley of d dams contains d reservoirs, each contributing one bias.

G. Trajectories and realized costs

The tables above measure the method against its own specification. The two figures of this subsection test it against two things no table reaches: whether the computed policy behaves like a water manager, and how large the quantity

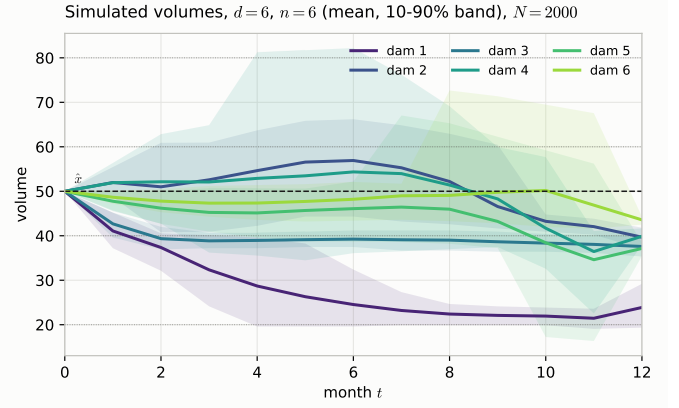


Fig. 2. Mean simulated volumes and 10–90% bands, $d = 6$, $n = 6$, $m = 4$, $N = 2000$. Dotted horizontal lines are the interior grid points.

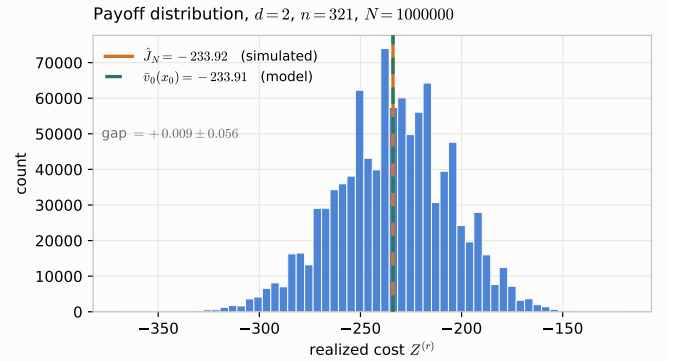


Fig. 3. Distribution of the realized costs $Z^{(r)}$ of (59), $d = 2$, $n = 321$, $N = 10^6$, with $\hat{J}_N$ and $\bar{v}_0(x_0)$ marked. The two differ by 0.009 on an axis with spread 180.

of Section IV-E is beside the variability the operator actually experiences.

Fig. 2 answers the first. The policy was given nothing but the costs (6)–(7) and the dynamics (4); what emerges is the strategy a manager would choose. It empties the valuable dam first, drawing dam 1 from 50 down to about 22 while the lower valley fills: a unit released at dam 1 is sold again at every dam on its way out, at dams 1, 2, 4, 5 and 6, earning 5.00 against the 0.70 earned by a unit already at the outlet.

It stores through the wet months – dams 2 and 4 peak near $t = 6$ – and releases through the autumn, when inflows no longer refill what is spent. And it ends every dam below the target, accepting the terminal cost (7), which at $\alpha^i = 0.1$ is mild beside what the water earns. The figure also makes the obstruction of Section IV-D visible rather than tabulated: the mean paths sit on the dotted interior grid points, six volumes per dam being all the policy has to choose between.

Fig. 3 answers the second, and measures the gap against something other than itself. The realized costs have a standard deviation of 28.52 and a single year ranges over roughly $[-330, -150]$, while the gap at this grid is 0.009, which is why $\hat{J}_N$ and $\bar{v}_0(x_0)$ fall on the same point of the axis. Two quantities are easily confused here, and the figure separates them: the confidence interval (61), of half-width 0.056, says how precisely N scenarios locate the *mean* cost, whereas the width of the histogram says how far a *single* year falls from that mean, and is about five hundred times larger. Refining the grid reduces neither. Only at six dams does the gap become visible on this scale, at 20.6 against a standard deviation of 179, which is what seven points per dam costs.

H. Comparison with Carpentier et al. ^[1]

A numerical comparison is not available. Their Table 1 reports a DP value of -3743 on the four-dam academic valley, but the instance data that produced it are not published, and the cost of a valley is not scale free: ours is -775 at $d = 4$ because our capacities, inflows and prices are different numbers. What can be compared is how the method behaves.

On tractability, their Table 1 gives a DP time of 1600 minutes at four dams and an extrapolated 10^8 minutes at six, from which they conclude that DP is implementable up to about five dams. We solve six dams in 110.5 minutes. This is not a contradiction and the reason is worth stating: their figure is for a state grid fine enough to serve as a reference value, ours is $n = 7$ points per reservoir, and Sections IV-E and IV-F measure what that coarsening costs. The frontier of exact dynamic programming is set by the mesh one is prepared to accept and not by d alone, and it is that trade which their paper leaves unquantified.

On the size of the error, the last line of their Table 1 gives a DADP–SDDP gap of 2.0% at four dams and 3.0% at six. Our relative gap, which measures the discretization alone, runs from 0.00% at $d = 2$, $\delta = 0.31$ to 3.62% at $d = 6$, $\delta = 33.33$, and is 1.32% at the finest six-dam mesh we can afford. At the resolutions a six-dam valley permits, discretizing therefore costs an error of the same order as the difference between their two methods. However, which route is cheaper for a given accuracy is not settled here as their instance and ours are different.

Their Fig. 7 reports the empirical payoff law of the four-dam valley under SDDP and DADP, obtained as ours is by simulating the strategy on 10^5 scenarios, and observes that the two methods agree closely in expectation while the shapes of the two distributions differ. Our Fig. 3 is the same diagnostic applied to the discretization, and carries the same warning: $\hat{J}_N$ summarizes a distribution whose standard deviation is about

12% of $|\hat{J}_N|$ in all three valleys, and two strategies with the same expected cost can differ in how bad their worst years are. The criterion (8) is an expectation, so it does not distinguish them.

V. LIMITATIONS AND CONCLUSION

THE instance is the first limitation. Carpentier et al. ^[1] publish no capacities, inflow laws, prices or targets for their academic valleys, so the numbers of Table I are ours and no value reported here can be set beside theirs. What Section IV-H compares is how the two methods behave: the resolution each can afford, and the size of the error each accepts.

Two simplifications are ours rather than theirs. The inflows are perfectly correlated across dams, a single multiplier (63) scaling every dam at once, so the valley is wet or dry as a whole. In the six-dam valley, whose two branches meet at dam 4, the branches never receive opposite inflows, so holding water in one to cover a shortfall in the other is a decision the policy is never faced with. And under the Hazard–Decision structure the month’s inflow is known before the release is chosen, which Section II-A1 noted is optimistic relative to an operator deciding daily ^[6]. Both make the valley easier to operate than a real one, so the values of Section IV are optimistic in level. Neither affects the comparisons made there, which hold the model fixed and vary only the discretization. The criterion (8) is in addition an expectation, with the consequence noted at the end of Section IV-H: it cannot distinguish two strategies of equal mean cost and unequal worst years.

The method’s own limit is the one measured in Section IV-D. The cost (62) is exponential in the number of dams, and the six-dam valley admits $n = 7$ points and $m = 4$ release levels per dam against $n = 1281$ and $m = 10$ at two dams. Both coarsenings are consequential, and Section IV-F measures the control grid as the larger of the two.

Within those limits, three things are established. Multilinear interpolation attains on 121 grid points the accuracy nearest-neighbour rounding reaches only on 103,041, and errs on the safe side where rounding does not: its value function understates what its own strategy earns, on every grid tested. The discretization error is small where the mesh is fine and not otherwise. Six dams are solvable exactly, in 110.5 minutes, provided one accepts a mesh of $\delta = 16.67$; the frontier of exact dynamic programming is therefore set by the resolution one is prepared to accept and not by d alone, which is the trade the decomposition methods are built to avoid making.

The formulation here is finite-horizon because the valley is: the seasonality of (47) and the terminal target of (7) both single out the twelve months. An infinite-horizon discounted criterion, solved by value iteration or policy iteration rather than by backward induction, would drop the terminal condition and ask instead for a stationary release rule. The discretization of Section II and its cost would carry over unchanged, as would the obstruction of Section IV-D; that is the natural continuation of this work.

REFERENCES

- [1] P. Carpentier, J.-Ph. Chancelier, V. Leclère, and F. Pacaud, “Stochastic decomposition applied to large-scale hydro valleys management,” *European Journal of Operational Research*, vol. 270, no. 3, pp. 1086–1098, 2018.
- [2] M. V. F. Pereira and L. M. V. G. Pinto, “Multi-stage stochastic optimization applied to energy planning,” *Mathematical Programming*, vol. 52, pp. 359–375, 1991.
- [3] K. Barty, P. Carpentier, and P. Girardeau, “Decomposition of large-scale stochastic optimal control problems,” *RAIRO – Operations Research*, vol. 44, no. 3, pp. 167–183, 2010.
- [4] M. Akian, “Markov Decision Processes: Dynamic Programming and Applications,” Lecture notes, ENSTA Course APM 50D2A/B & M2 “Optimization, Games, and Control,” Institut Polytechnique de Paris and Paris-Saclay University, 2026.
- [5] D. P. Bertsekas, *Dynamic Programming and Optimal Control*, vol. I, 4th ed. Belmont, MA, USA: Athena Scientific, 2017.
- [6] A. Street, D. Valladão, A. Lawson, and A. Velloso, “Assessing the cost of the hazard-decision simplification in multistage stochastic hydrothermal scheduling,” *Applied Energy*, vol. 280, Art. no. 115939, 2020.
- [7] C.-S. Chow and J. N. Tsitsiklis, “The complexity of dynamic programming,” *Journal of Complexity*, vol. 5, no. 4, pp. 466–488, 1989.
- [8] G. J. Gordon, “Stable function approximation in dynamic programming,” in *Proc. International Conference on Machine Learning (ICML)*, 1995, pp. 261–268.